

Assignment 1

Build a Multi-Agent Stock Research Desk with LangGraph

Supervisor + 3 specialist agents · shared state · conditional routing · tool calling

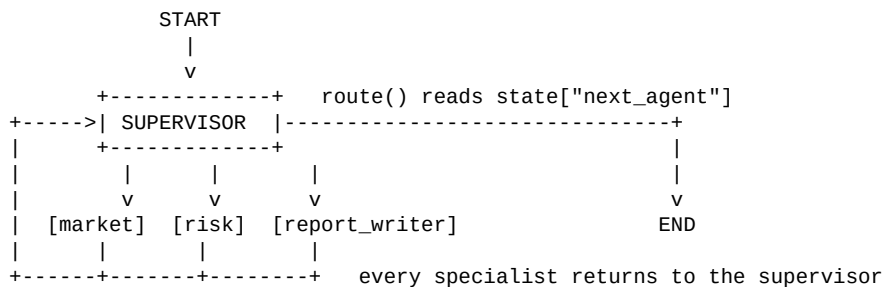
Item	Details
Level	Beginner to intermediate (you know Python and have seen a basic LangGraph graph)
Time	3 to 5 hours
Stack	Python 3.10+, LangGraph, LangChain-OpenAI, yfinance
You submit	Your code folder + an answers file for every question in <i>Questions.pdf</i>
Reference project	<code>financial_mas_1/financial_mas</code> (the class demo). This assignment is a smaller version of it.

1. What you are building

A user types a **stock ticker** and a **question** in plain English, for example *"How risky is NVIDIA compared to a typical stock?"*. A small team of AI agents then works together, like a research desk at a bank, to answer it with real market data.

Agent	Uses an LLM?	Job	Writes to state
Supervisor	No (plain Python)	Looks at what is still missing and decides which agent runs next.	next_agent
Market Data Agent	Yes, with tools	Lets the LLM call <code>get_stock_price</code> and <code>get_company_info</code> (the ReAct loop).	market_data
Risk Analyst Agent	Yes	Runs a risk calculation in Python, then asks the LLM to explain the numbers.	risk_metrics, risk_summary
Report Writer Agent	Yes	Reads everything the team found and answers the user's question.	answer

The graph (hub and spoke)



Each specialist does one job and hands control back. The supervisor is the only node that decides where to go next. This is the same pattern as the class demo, with one agent (the Fundamental Analyst) removed to keep it small.

What you will learn

- Designing a **shared state** (a TypedDict) that every agent reads and updates.
- Using a **reducer** (`Annotated[list, operator.add]`) so agents append to a log instead of overwriting it.
- Writing **nodes** that return a *partial* update instead of the whole state.
- Routing with **add_conditional_edges** and a supervisor.
- Giving an LLM **tools** and running the tool-calling (ReAct) loop yourself.
- Keeping the LLM honest: real data in, cited numbers out.

2. Setup

Step 2.1: Create the folder and a virtual environment

```
mkdir stock_desk && cd stock_desk
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install langgraph langchain-openai yfinance python-dotenv
```

Step 2.2: Add your OpenAI key

Create a file named `.env` in the folder:

```
OPENAI_API_KEY=sk-...your-key...
LLM_MODEL=gpt-4o-mini
```

Never commit `.env` to GitHub. Add it to `.gitignore` straight away.

Step 2.3: Project structure

```
stock_desk/
|-- .env           # your API key (never commit this)
|-- state.py       # Step 3: the shared state schema
|-- tools.py       # Step 4: functions that fetch real data
|-- agents.py      # Step 5: supervisor + 3 specialist agents
|-- pipeline.py    # Step 6: wires the agents into a LangGraph
|-- main.py        # Step 7: run it from the terminal
`-- answers.md     # Step 8: your answers to Questions.pdf
```

File	What goes in it	Maps to (class demo)
state.py	StockDeskState TypedDict	core/config.py
tools.py	get_stock_price, get_company_info, calculate_risk_metrics	tools/financial_tools.py
agents.py	supervisor, route, market_data_agent, risk_analyst_agent, report_writer_agent	agents/*.py
pipeline.py	build_graph(), ask()	graph/pipeline.py
main.py	Command-line entry point	main.py

3. The shared state (state.py)

The state is the **baton** passed between agents. Every node receives the whole state and returns **only the keys it changed**. LangGraph merges that update in for you.

Most keys are simply *replaced* when a node returns them. `steps` is different: `Annotated[list[str], operator.add]` tells LangGraph to **append** the new list to the old one. That gives you a free audit trail of every agent's action.

```
"""state.py - the shared state every agent reads from and writes to."""
import operator
from typing import Annotated, Any, TypedDict

class StockDeskState(TypedDict):
    # --- Input ---
    ticker: str                # e.g. "AAPL"
    question: str              # the question the desk must answer

    # --- Filled by agents ---
    market_data: dict[str, Any] # Market Data Agent
    risk_metrics: dict[str, Any] # Risk Analyst Agent
    risk_summary: str           # Risk Analyst Agent
    answer: str                 # Report Writer Agent

    # --- Control ---
    next_agent: str             # set by the Supervisor

    # --- Audit trail: APPENDED by every agent, never overwritten ---
    steps: Annotated[list[str], operator.add]
```

Check your understanding

- What would happen to `steps` if you removed the `Annotated[... , operator.add]`?
- Why does the supervisor not need its own output key apart from `next_agent`?

4. The tools (tools.py)

Agents decide; tools do the work. We use **yfinance** (free Yahoo Finance data, no key needed).

Function	@tool?	Why
get_stock_price	Yes	The LLM chooses when to call it, so it must carry a docstring and type hints the LLM can read.
get_company_info	Yes	Same reason as above.
calculate_risk_metrics	No	The Risk Analyst always needs it, so we call it straight from Python. Not everything has to be an LLM decision.

The risk maths, in one line each: volatility = std-dev of daily returns $\times \sqrt{252}$ (trading days); max drawdown = the worst fall from a previous peak; 1-year return = (last price – first price) / first price.

Every tool catches its own errors and returns {"error": ...} instead of crashing, so one bad ticker cannot bring down the whole graph.

```
"""tools.py - plain Python functions the agents use to get real data."""
import math
import yfinance as yf
from langchain_core.tools import tool

@tool
def get_stock_price(ticker: str) -> dict:
    """Get the current price, daily change and 52-week range of a stock."""
    try:
        info = yf.Ticker(ticker.upper()).fast_info
        price, prev = float(info.last_price), float(info.previous_close)
        return {
            "ticker": ticker.upper(),
            "current_price": round(price, 2),
            "change_pct": round((price - prev) / prev * 100, 2),
            "week_52_high": round(float(info.year_high), 2),
            "week_52_low": round(float(info.year_low), 2),
            "market_cap_bn": round(float(info.market_cap) / 1e9, 1),
        }
    except Exception as e:
        return {"ticker": ticker, "error": str(e)}

@tool
def get_company_info(ticker: str) -> dict:
    """Get the company name, sector and industry of a stock."""
    try:
        info = yf.Ticker(ticker.upper()).info
        return {
            "company_name": info.get("longName", ticker),
            "sector": info.get("sector", "Unknown"),
            "industry": info.get("industry", "Unknown"),
            "currency": info.get("currency", "USD"),
            "pe_ratio": info.get("trailingPE"),
        }
    except Exception as e:
        return {"ticker": ticker, "error": str(e)}

def calculate_risk_metrics(ticker: str) -> dict:
    """Annualised volatility, max drawdown and 1-year return from 1 year of prices."""
    try:
        prices = yf.Ticker(ticker.upper()).history(period="1y")["Close"].tolist()
        if len(prices) < 20:
            return {"ticker": ticker, "error": "Not enough price history"}

        returns = [(prices[i] - prices[i - 1]) / prices[i - 1] for i in range(1, len(prices))]
        mean = sum(returns) / len(returns)
        daily_std = math.sqrt(sum((r - mean) ** 2 for r in returns) / (len(returns) - 1))
        volatility = daily_std * math.sqrt(252)  # 252 trading days a year

        peak, max_dd = prices[0], 0.0
        for p in prices:
            max_dd = max(max_dd, p - peak)
            peak = p
    except Exception as e:
        return {"ticker": ticker, "error": str(e)}
```

```

        peak = max(peak, p)
        max_dd = max(max_dd, (peak - p) / peak)

    return {
        "ticker": ticker.upper(),
        "volatility_pct": round(volatility * 100, 2),
        "max_drawdown_pct": round(max_dd * 100, 2),
        "one_year_return_pct": round((prices[-1] - prices[0]) / prices[0] * 100, 2),
        "risk_level": "HIGH" if volatility > 0.40 else "MEDIUM" if volatility > 0.20 else "LOW",
    }
except Exception as e:
    return {"ticker": ticker, "error": str(e)}

```

5. The agents (agents.py)

Every agent is just a Python function: state in -> partial dict out. Read the code top to bottom; each agent is explained below it.

```
"""agents.py - the Supervisor and the three specialist agents."""
import json
import os

from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage, SystemMessage, ToolMessage

from state import StockDeskState
from tools import get_stock_price, get_company_info, calculate_risk_metrics

load_dotenv()
MODEL = os.getenv("LLM_MODEL", "gpt-4o-mini")

# -----
# 1. SUPERVISOR - no LLM. Looks at what is missing and picks the next agent.
# -----
def supervisor(state: StockDeskState) -> dict:
    if not state.get("market_data"):
        nxt = "market_data_agent"
    elif not state.get("risk_metrics"):
        nxt = "risk_analyst_agent"
    elif not state.get("answer"):
        nxt = "report_writer_agent"
    else:
        nxt = "END"
    return {"next_agent": nxt, "steps": [f"SUPERVISOR -> {nxt}"]}

def route(state: StockDeskState) -> str:
    """Used by add_conditional_edges: returns the name of the next node."""
    return state["next_agent"]

# -----
# 2. MARKET DATA AGENT - an LLM that decides which tools to call (ReAct loop).
# -----
MARKET_TOOLS = {"get_stock_price": get_stock_price, "get_company_info": get_company_info}

def market_data_agent(state: StockDeskState) -> dict:
    llm = ChatOpenAI(model=MODEL, temperature=0).bind_tools(list(MARKET_TOOLS.values()))
    messages = [
        SystemMessage("You are a market data agent. Call BOTH tools for the ticker. "
                       "Never invent numbers."),
        HumanMessage(f"Fetch market data for {state['ticker']}"),
    ]
    data = {}
    for _ in range(4):
        response = llm.invoke(messages)
        if not response.tool_calls:
            # LLM is done calling tools
            break
        messages.append(response)
        for call in response.tool_calls:
            result = MARKET_TOOLS[call["name"]].invoke(call["args"])
            data.update(result)
            messages.append(ToolMessage(json.dumps(result), tool_call_id=call["id"]))
    return {"market_data": data,
            "steps": [f"MARKET_DATA_AGENT: price={data.get('current_price')}"]}

# -----
# 3. RISK ANALYST AGENT - runs the calculation, then the LLM interprets it.
# -----
def risk_analyst_agent(state: StockDeskState) -> dict:
    metrics = calculate_risk_metrics(state["ticker"])
    llm = ChatOpenAI(model=MODEL, temperature=0.1)
    summary = llm.invoke([
        SystemMessage("You are a risk analyst. In 3 short bullet points, explain "
```

```

        "what these numbers mean for an investor. Quote the numbers."),
        HumanMessage(json.dumps(metrics)),
    ]).content
    return {"risk_metrics": metrics, "risk_summary": summary,
            "steps": [f"RISK_ANALYST_AGENT: risk_level={metrics.get('risk_level')}"]}

# -----
# 4. REPORT WRITER AGENT - answers the user's question from everyone's work.
# -----
def report_writer_agent(state: StockDeskState) -> dict:
    llm = ChatOpenAI(model=MODEL, temperature=0.3)
    context = (
        f"MARKET DATA: {json.dumps(state['market_data'])}\n"
        f"RISK METRICS: {json.dumps(state['risk_metrics'])}\n"
        f"RISK ANALYST NOTES: {state['risk_summary']}"
    )
    answer = llm.invoke([
        SystemMessage("You are the head of a stock research desk. Answer the user's "
                      "question using ONLY the data given. Cite numbers. If the data "
                      "cannot answer it, say so. End with one line: "
                      "'Verdict: BUY / HOLD / SELL' and a not-financial-advice note."),
        HumanMessage(f"QUESTION: {state['question']}\n\n{context}"),
    ]).content
    return {"answer": answer, "steps": ["REPORT_WRITER_AGENT: answer written"]}

```

How each agent works

Supervisor and route()

A **waterfall**: the first missing piece decides the next agent. No LLM is needed, which makes routing fast, free and predictable. `route()` is *not* a node; LangGraph calls it after the supervisor to pick an edge, and its return value must match a key in the routing map.

Market Data Agent: the ReAct loop

- `bind_tools()` tells the LLM which tools exist.
- The LLM answers with `tool_calls` (which tool, which arguments).
- **We** run each tool and send the result back as a `ToolMessage` carrying the same `tool_call_id`.
- Repeat until the LLM stops asking for tools. The `range(4)` is a safety limit against endless loops.

Risk Analyst Agent

Python computes the numbers (reliable); the LLM explains them (readable). Never ask an LLM to do arithmetic you can do in code.

Report Writer Agent

Gets the full context and the user's question. The prompt forces it to use **only** the data given, cite numbers, and admit when the data cannot answer the question. Several questions in Questions.pdf test exactly this.

6. Building the pipeline (pipeline.py)

This is where the agents become a graph. Building any LangGraph comes down to five calls:

#	Call	Meaning
1	StateGraph(StockDeskState)	Create a graph that knows the state schema and its reducers.
2	add_node(name, fn)	Register each agent. The name is what edges and route() refer to.
3	add_edge(a, b)	Fixed path: START → supervisor, and every specialist → supervisor.
4	add_conditional_edges(src, fn, map)	After the supervisor, call route(); map its return value to a node or END.
5	compile()	Check the wiring and return a runnable graph (.invoke / .stream).

```
"""pipeline.py - wires the agents into a LangGraph (hub-and-spoke)."""
from langgraph.graph import StateGraph, START, END

from state import StockDeskState
from agents import (supervisor, route, market_data_agent,
                    risk_analyst_agent, report_writer_agent)

def build_graph():
    graph = StateGraph(StockDeskState)

    # 1. Nodes
    graph.add_node("supervisor", supervisor)
    graph.add_node("market_data_agent", market_data_agent)
    graph.add_node("risk_analyst_agent", risk_analyst_agent)
    graph.add_node("report_writer_agent", report_writer_agent)

    # 2. Entry point
    graph.add_edge(START, "supervisor")

    # 3. Every specialist reports back to the supervisor
    for agent in ["market_data_agent", "risk_analyst_agent", "report_writer_agent"]:
        graph.add_edge(agent, "supervisor")

    # 4. Supervisor decides where to go next
    graph.add_conditional_edges("supervisor", route, {
        "market_data_agent": "market_data_agent",
        "risk_analyst_agent": "risk_analyst_agent",
        "report_writer_agent": "report_writer_agent",
        "END": END,
    })

    return graph.compile()

def ask(ticker: str, question: str) -> dict:
    initial_state = {
        "ticker": ticker.upper(), "question": question,
        "market_data": {}, "risk_metrics": {}, "risk_summary": "",
        "answer": "", "next_agent": "", "steps": [],
    }
    return build_graph().invoke(initial_state)
```

Common bug: if route() returns a string that is not a key in the map, LangGraph raises KeyError. For example, returning "auto_approve" when the map key is "approve". Keep node names in one place.

How the state changes during one run

```
t0 initial state          market_data={} risk_metrics={} answer=""
t1 supervisor            next_agent = market_data_agent
t2 market_data_agent     market_data = {current_price, sector, ...}
t3 supervisor            next_agent = risk_analyst_agent
t4 risk_analyst_agent     risk_metrics = {...}, risk_summary = "..."
t5 supervisor            next_agent = report_writer_agent
t6 report_writer_agent    answer = "..."
t7 supervisor            next_agent = END -> graph stops, invoke() returns final state
```

7. Run it (main.py)

"""main.py - run the stock desk from the terminal.

Usage:

```
python main.py --ticker AAPL --question "Is Apple a good buy right now?"
```

"""

```
import argparse
```

```
from pipeline import ask
```

```
parser = argparse.ArgumentParser()
```

```
parser.add_argument("--ticker", default="AAPL")
```

```
parser.add_argument("--question", default="Should I buy this stock today?")
```

```
args = parser.parse_args()
```

```
result = ask(args.ticker, args.question)
```

```
print("\n=== AGENT STEPS ===")
```

```
for s in result["steps"]:
```

```
    print(" ", s)
```

```
print("\n=== ANSWER ===")
```

```
print(result["answer"])
```

```
python main.py --ticker NVDA --question "How risky is NVIDIA compared to a typical stock?"
```

Expected output (your numbers will differ because the data is live):

```
=== AGENT STEPS ===
```

```
SUPERVISOR -> market_data_agent
```

```
MARKET_DATA_AGENT: price=239.24
```

```
SUPERVISOR -> risk_analyst_agent
```

```
RISK_ANALYST_AGENT: risk_level=MEDIUM
```

```
SUPERVISOR -> report_writer_agent
```

```
REPORT_WRITER_AGENT: answer written
```

```
SUPERVISOR -> END
```

```
=== ANSWER ===
```

```
NVIDIA (NVDA) has a volatility of 37.82% ... maximum drawdown of 20.21% ...
```

```
one-year return of 29.6% ... Overall, NVDA is categorized with a medium risk level.
```

```
Verdict: HOLD. This is not financial advice.
```

Troubleshooting

Symptom	Fix
Script prints nothing	Make sure main.py actually calls ask() and prints the result.
KeyError: 'xyz' from _branch.py	route() returned a name that is not in the add_conditional_edges map.
openai.AuthenticationError	.env is missing or has the wrong key; run from the folder that contains .env.
LangChainPendingDeprecationWarning	Harmless library warning. Hide it with: python -W ignore main.py
Graph loops forever / recursion limit	An agent is not filling its state key, so the supervisor keeps sending work back to it.

8. Tasks, deliverables and grading

Core tasks (required)

- **Task 1:** Build all five files and get one successful run.
- **Task 2:** Run every question in *Questions.pdf*. Paste the agent steps and the answer for each into `answers.md`.
- **Task 3:** Under each answer, write 1–2 lines of your own: is the answer correct, does it cite numbers, did it make anything up?
- **Task 4:** Draw the graph yourself: `print(build_graph().get_graph().draw_mermaid())` and paste the output into `answers.md`.

Extension tasks (pick at least one)

- **A. Add a 4th agent.** Create a *Fundamental Analyst* that fetches P/E, profit margin and debt/equity (see `fetch_fundamental_data` in the class demo). Add it to the state, the supervisor waterfall and the routing map.
- **B. Streaming.** Replace `invoke()` with `graph.stream(...)` and print each agent's update as soon as it finishes.
- **C. Two tickers.** Let the user pass two tickers and have the report writer compare them (Questions Part D).
- **D. Smart supervisor.** If the question is only about price, skip the Risk Analyst. Hint: the supervisor can read `state['question']`.

Submit

- Your `stock_desk/` folder **without** `.env` and `.venv`.
- `answers.md` with every question answered.

Grading (100 marks)

Area	Marks	What we look for
Graph runs end to end	25	START → supervisor → 3 agents → END with no errors
State and reducer used correctly	15	Partial updates; steps log is appended, not overwritten
Tool-calling loop	15	ToolMessage with <code>tool_call_id</code> ; loop has a safety limit
Questions answered	30	All questions run; answers cite real numbers; edge cases handled
Your own review (Task 3)	5	Honest notes on where the answer was right or wrong
Extension task	10	Works and is wired into the graph properly